

Lec 30 Practical Exercise - 3

Lec 30 Practical Exercise - 3

Lecture	30 / 63 + intro
Duration	35:13
Captions	English (auto-generated)
Words	5,113
Video	9TlyLSpTq4Y
Playlist	Fundamentals of Generative AI and LLMs

Auto-generated YouTube captions, lightly cleaned (removed [music] tags, grouped into paragraphs). These are **verbatim lecture captions**, not edited notes.

Transcript

[00:00] Hello all. Welcome to the hands-on series of fundamentals of generative AI and large language models theory and practice. In today's hands-on session, we'll be implementing the beta variational autoencoder, which is a type of variational autoencoder uh with the help of beta, that is a hyperparameter, we'll be able to get a trade-off between the reconstruction loss and the KL divergence term, which is present in the total loss function of a variational autoencoder.

[00:32] In the previous hands-on session, we have seen that the total loss incurring in a variational autoencoder comprises of both the reconstruction loss as well as the KL divergence term. So, this beta, when multiplied with the KL divergence term, brings a trade-off between the these two losses. So, in a traditional variational autoencoder, we see that the latent space distribution tries to be very close to the prior distribution. So, in the process of being very close to this prior distribution, the latent space becomes very much constrained.

[01:07] So, in the process of getting the latent space distribution close to the prior distribution, the limitation of finding the complex patterns in the data will be very much hindered in case of traditional variational autoencoder. So, to address this issue, we introduce something called as beta variational autoencoder, which brings the

trade-off between them. So, let us see the hands-on. So, initially, importing the necessary libraries. So, we import the TensorFlow as TF. TensorFlow is a deep learning framework, which is used to build the neural networks. Then, coming to TensorFlow Keras, we import the layers as well as model. So, layers like input layers, the hidden layers, as well as the output layers are all

[01:58] called from this function called as layers. And then we have to create the beta variational autoencoder, so that's why we need this function called as model. Then we import the numerical Python as np. Since we have to deal with numbers, integers, or arrays, we need this numerical Python. Then we import the matplotlib.pyplot as plt. So, this matplotlib is basically used for data visualization purpose. So, these are some of the important libraries that we have imported.

[02:32] Now coming to loading the MNIST data set. So, in the previous hands-on, we have seen how the different types of autoencoders, the types of variational autoencoders, or conditional variational autoencoders, all these were implemented using the MNIST data set. So, to have a better comparison of the performance of all these types of autoencoders, we once again take this MNIST data set, which is a handwritten digit data set comprising of 10 different numbers varying from 0 to 9.

[03:03] So, we load this MNIST data set into two different variables called as training as well as test variables. But in case of training, we take only the input variables, which is defined within the variable X train. And in case of test, we consider only the X test as one of the variable where we have all the input test samples. And this underscore refers to ignorance of the Y, that is the target variable. So, in case of variational autoencoder, the use of target variable is not very much necessary. It is used only in case of the conditional variational autoencoder.

[03:43] That's why we ignore the output Y. So, that's why we are representing two underscores both in the training as well as the test set. Now, converting this data type into floating data type and then normalization of the data. So, since we know that the MNIST data set is a grayscale data, wherein the pixel values vary from 0 to 255. The maximum value is 255. Therefore, to have all the values to range between 0 and 1, we have this division by 255. So, suppose we take an example of an image, say a 3 cross 3 image.

[04:30] The pixel values, because that it is a grayscale, there is only one channel and the pixel values vary from 0 to 255. Say, the values are this way. So, what we do is we do 5 by 255. We do 10 by 255, 0 by 255. So, the minimum value is 0, so for which we get the minimum value as 0 after normalization. Uh suppose we in place of two, if we have 255 here, the division happens to be 255 by 255. So, the maximum possible range is 0 to 1. So, the normalized values will be in the range 0 to 1.

[05:22] So, as mentioned earlier, we do not take the target variable or the output label. That is why we reshape the training as well as the test data, so that we have

only the input variables. That's why we write this minus one and we take all the 784 input features or input variables into consideration. Uh as mentioned in the previous hands-on, this MNIST data set is a 28 cross 28 pixel data. We have 28 pixels in width and 28 pixels in height. So, when we multiply 28 into 28, we get this 784 input variables. That's why the number 784 here.

[06:00] So, let us see with the help of print statement what exactly is the shape of the training set as well as the test set. So, the output output shows that we have 60,000 samples in the training set and there are 784 input variables. Similarly, there are 10,000 samples in the test set and 784 input variables. So, this shows the shape of the training as well as the test data. Now, coming to defining some of the parameter. So, the highlight of this beta variational autoencoder is the use of beta values. So, beta values, as we have seen in the theory sessions, beta value is always a non-negative value. It can be any value which is greater than or equal to zero. There is no restriction on the upper limit.

[06:53] So, we can define beta as a non-negative value. That is, beta will be greater than or equal to zero. So, in the theory sessions, we have already seen what happens as and when we increase the value of beta. Beta and its impact or effect. When beta value will be equal to one, it is nothing but a traditional variational autoencoder. Why? Because the total loss of a variational autoencoder comprises of two components. That is, the reconstruction loss denoted as RL and the KL divergence loss that is represented as KL.

[07:41] So, in case of a beta variational autoencoder, we have this reconstruction loss plus this beta, which is a scaling factor, will be multiplied with the KL divergence term. So, with the help of this beta, we are trying to have a trade-off between the reconstruction loss and the KL divergence term. So, beta is a non-negative value. So, when beta value will be equal to one, the equation turns out to be RL plus KL, which is the standard equation for VAE.

[08:17] So, that's why we have written when beta is equal to one, it is the traditional VAE. Now, what if beta is greater than one? Say, for example, beta is equal to two or three. Then, there will be more this entanglement. And what if beta value is less than one? It is less this entanglement. Entanglement uh refers to the space using which uh the complex patterns between the data will be understood. So, as and when we increase the value of beta, uh there will be more weightage given to the KL divergence. So, with the help of KL divergence, it is uh it is very easy to understand the complex patterns or relationship between the latent variables present in the data. So, but when as and when we increase the value of beta, maybe beta

[09:17] is 100 or 2,000. Likewise, when we increase, it goes beyond a saturation level, then this is this value will be at loss. That is, the reconstruction loss will be very, very higher. So, finding the optimal value of beta is very much challenging. So, that

is why we make a comparative study of different beta values in our experimentation. So, coming back to the number of parameters that we have to define. So, as we have seen in the previous hands-on, the latent space that is considered in a variational autoencoder, that is Z , is dependent on two parameters. That is the mean as well as the variance. We have already seen it.

[10:02] So, the latent space Z will be equal to μ plus standard deviation into a small noise, that is ϵ . So, here μ refers to mean, σ refers to the standard deviation, and ϵ refers to a random noise. Usually, this random noise is a Gaussian noise. Gaussian noise with zero mean and one standard deviation. So, now latent dimension refers to the number of latent variables that we really want from the 784 different variables. So, we have seen that the input data comprises of 784 different input variables. Now, these will be down-sampled into 10 different latent variables.

[10:55] And we have to now consider the beta values so that we get a trade-off between the two different loss terms. So, let us have an experiment of five different beta values. That's why we start from beta as a 0.5, then take 1, 2, 4, and 10. So, we gradually increase the values of beta and see their performance. Coming to some more hyper parameters, epochs. Let the number of epochs be five. Reduce the number of epochs as five because we have to run this for five different beta values. So, five different models we have to train.

[11:29] That's why we I have restricted the value of epochs as five itself, but user is always free to experiment with the number of epics. And now coming to the batch size as 128. So, once again, we are free to use the number of training samples that we want in a batch. So, I have considered 128 different samples, after which we want to update the weights. So, batch size is equal to 128. So, this is the initial parameter taking of the taking the parameter values.

[12:02] Now coming to defining the sampling layer. So, variational autoencoder comprises of three layers, the encoder, the sampling layer, and the decoder. So, the encoder takes in the input variables, that is 784 input variables, and at the sampling layer, we create one latent space, that is Z , which is dependent on the mean as well as the standard deviation, and that is multiplied with some random noise. So, we have to define what is mean, what is variance, and how do we get it from the inputs.

[12:32] So, sampling creation of a sampling layer here. So, inputs. From the inputs, we get two different terminologies, that is the mean and the variance. So, from the variance, we'll be able to get the standard deviation. So, in the previous hands-on, we have already derived the relationship between the standard deviation and the variance. Why exactly we write this as log variance? It is already explained in the previous hands-on of variational autoencoder.

[13:01] So, now what is epsilon? Epsilon is a random noise, which is a normally distributed noise or a Gaussian noise. And shape of such a noise is same as the shape

of the mean values. So, we have 16 different latent variables. Therefore, we will when we take the mean, we will get 16 different mean values. Similarly, we can get 16 different standard deviations or 16 different uh 16 different variance values. So, now the epsilon value is dependent on the shape of the mean values. So, that's why we write shape as same as that of the mean values.

[13:40] So, all together we have to return the mean value, that is the latent space will consider the mean plus this is the standard deviation term $e^{\text{power } 0.5}$ into the variance term $\log$ of the variance term into the epsilon. So, this is nothing but the fundamental equation or the mathematically represent equation that is written here $z = \mu + \sigma \cdot \epsilon$. So, this is all about the sampling layer. Now, coming to the encoder function, how do we create this encoding part? So, encoder encoder makes use of the inputs, that is 784. The shape is 784 input variables. Then we have the fully connected layer, which makes use of 256 neurons and the activation function is ReLU. ReLU adds some non-linearity to the data. It will

[14:35] convert all the negative values into positive values because the functioning of a ReLU is in such a way that ReLU will be $\max(0, x)$. Suppose we take some negative value say minus 10, then this equation turns out to be maximum of 0 {comma} minus 10. So, maximum of these two values will be equal to 0. Now, if we take some positive value say 100. So, this will be maximum of 0 {comma} 100. So, therefore maximum value is 100. So, this adds some non-linearity to the data.

[15:13] So, we have this ReLU activation function. All together, these are acting on the the encoder inputs, which is defined here. So, we have a fully connected layer in order to get the mean value. We have another fully connected layer, which is associated with the variance value. And we do sampling so that we get a random noise generation, which which is got from the mean as well as the variance of the data. So, Z will be obtained by sampling of the mean as well as the variance of the data or standard deviation of the data. So, all together, the encoder architecture is a model which comprises of the encoder inputs, which is already defined here. These are the encoder inputs. Also, the mean value, the standard deviation, and the

[16:06] Z, which is a flexible vector, which is dependent on mean as well as the variance or the standard deviation of the data. Now, coming to the decoder architecture. Decoder, completely opposite to encoder. So, to the decoder, what goes as input is a latent dimension. So, we a latent dimension defined here will is equal to 10. So, the dimension is 10 here that goes as input or shape to this set of layers. Soon after which, we have the dense layer or the fully connected layer having 256 different neurons and the same ReLU activation function to add the non-linearity to the data.

[16:47] Then, we have fully connected layer. Uh since we have to reconstruct back the original image, we need to have the 784 number of input variables. That's why we have to connect this set of 784 neurons. Now, the activation function here what

we have is sigmoid activation function. So, most of the we have already seen what is the importance of a sigmoid activation function. So, sigmoid activation function, sigmoid of x is $1 / (1 + e^{-x})$.

[17:21] x could be any value varying from minus infinity to infinity. When it varies from minus infinity to infinity, values will vary from zero to one. We have to just substitute the value of x as minus infinity and infinity. So, the minimum value and the maximum value that we obtain is zero and one, which is nothing but a probability of values that that we get or the normalized values what we have already seen during the pre-processing of the data. That is why deliberately we are taking this sigmoid activation function.

[17:58] So, altogether, the decoder architecture is a model which makes use of the latent inputs. In our case, the latent dimension will be equal to 10. And the outputs, the outputs are the 784 input variables that we want. And finally, the 784 input values that we get or the pixel values that we get have to be reshaped so that we get 28 cross 28. We get a 2D array or nothing but the reconstruction of an image. So, this is all about the decoder architecture.

[18:30] Now, coming to creation of the beta variational autoencoder. So, we create this beta variational autoencoder from the TensorFlow model class. So, this comprises of the encoder, the decoder, and the various beta values. `Uh` is a hyperparameter which is introduced in this kind of variational autoencoder. So, this is super is `uh` super calls the parent class methods. It initializes the TensorFlow model correctly. So, that's why this super term is used.

[19:10] So, we have this encoder architecture from the encoder-decoder which is we have already defined and beta the set of five beta values which we have already considered. So, now coming to tracking of the total loss. So, the highlight of this variational autoencoder is getting the total loss which is dependent on the beta term. So, what does the total loss comprise of? It comprises of the reconstruction loss. It comprises of the KL loss. So, what is this total loss? It is a `uh` metric which is used for understanding the performance of the variational autoencoder or beta variational autoencoder. Let the name given to this loss is this total loss is `loss`. So, that while displaying `uh` the values after each epoch, we want the

[19:56] term as `loss`. Same thing, let us track all the reconstruction loss for different beta values. Let the name given to this loss be reconstruction loss. So, both the loss are obtained from the TensorFlow Keras and the `uh` sub package called as metrics. Same thing to do with the KL divergence term. Let us also track this loss from the TensorFlow Keras metric sub package and the `uh` and the function used is `mean`. So, let the name given to this loss is KL loss.

[20:31] Why do we take this mean value is so that we get the average loss. We are not considering an outlier kind of loss with too high noise or too high loss or too low

loss. In order to avoid that high value or a low value, we just take the mean of all the loss terms. That's why we need to have this uh function called as mean here. Now, we have this at property. It makes a method behave like an attribute. So, it allows the TensorFlow to access the metrics automatically. That's why we have this property here using which we define the different metrics like total loss tracker, reconstruction loss tracker and the KL divergence loss tracker.

[21:15] Now coming to training part of the model. So as and when we train, we consider we do some prediction. On the prediction we consider the loss term. From the loss term we get the gradients. From the gradients we update the weights. So gradients will help us understand how much weights have to be updated whether the weights have to be increased or decreased. So the relationship between all of them can be given as new weight will be equal to the old weight minus the learning rate of the model into $\frac{\partial L}{\partial W}$. So this $\frac{\partial L}{\partial W}$ gives us the gradient. So it will help us identify the whether the weights have to be increased or decreased.

[22:01] And to what extent it has to be increased or decreased. So to the encoder what data we take is the latent space vector that is Z which is dependent on the mean and the standard deviation or the variance of the data and to decoder whatever input that goes is the latent space vector which is Z . Reconstruction loss will be equal to the binary cross entropy loss. So total loss is equal to reconstruction loss plus beta into the KL divergence loss. Now what is this reconstruction loss? Let us consider the binary cross entropy loss which is equal to $-\sum Y \log(Y) - (1-Y) \log(1-Y)$. Y refers to the true label and $\hat{Y}$ refers to the predicted output or the original output and the predicted output.

[23:17] This is how we have the binary cross entropy loss. Now coming to the KL divergence loss, which will be equal to $-\frac{1}{2} \sum (1 + \log(\frac{\sigma^2}{\mu^2}) - \frac{\mu^2}{\sigma^2})$. This is the KL divergence loss. So this KL divergence loss is multiplied by the scaling factor, which is beta. So we have the reconstruction loss, which is the binary cross entropy loss. As mentioned earlier, we always reduce the loss by taking the mean of the loss.

[24:01] That's why we have this mean function here. Similarly, KL divergence loss, which is $-\frac{1}{2} \sum (1 + \log(\frac{\sigma^2}{\mu^2}) - \frac{\mu^2}{\sigma^2})$. So total loss will be reconstruction loss plus beta into KL loss. So this is the only difference between the variational autoencoder, which we have seen in the previous hands-on, and the beta variational autoencoder. So based on this loss, we calculate the gradient. So we have already seen the relationship between loss and the gradient. So based on the gradient term, we update the weights what during the back propagation how what is the weight updation to be done.

[24:47] So for which we can make use of any optimizer. In our case, we try to always maintain this Adam optimizer. Now what is extra here is tracking and updating of the

state. Why we need to track is since we have taken five different beta values, we need to make use of this tracker so that updation of loss happens for different beta values because we have to enumerate or accumulate all the beta values. We have to store it so that we have a history of different losses pertaining to different beta values and then we have need to have a plot of it. That's why we have this tracker. So, tracker is like like a running calculator. It stores the values.

[25:29] So, we have a tracker for updating the total loss. We have a tracker for updating the reconstruction loss. Same thing, tracker for updating the KL loss. So, altogether we written three different loss values. That is the reconstruction loss, the KL loss, and the total loss. So, this is how we have created the beta variational autoencoder. Now, coming to training the different beta values, training the model with different beta values.

[25:58] For which we can we take empty list. One list consisting of reconstruction loss, another list for having the KL losses. And for trained models, we have this dictionary. Always a dictionary is made up of a key and a value. We need two values here because beta can be considered as key here and whatever model we have. For the first beta value, we can consider the first model. For the second beta value, we have the second model.

[26:27] So, likewise for five different beta values, we need five different models. That's why we have a key which is a set of beta values and the model which is the value. That's why we are taking one empty dictionary here. So, set of empty list and set of and a dictionary here. So, for different beta values, we let us print the training in the during the training, let us print the loss as well as different beta values. So, how do we have the architecture? We have to create one encoder, we have to create a decoder, as well as have a set of beta values. So, our model beta variational autoencoder comprises of encoder, decoder, and set of beta values.

[27:11] And then coming to configuring the model, so we make use of the Adam optimizer. In each of the cases, we make use of the Adam optimizer. And then we train the model on the `X_train` values. Let the number of epochs be equal to five, as mentioned earlier. Batch size we have already taken as 128, and let the verbose be equal to one. So, let there be let there be one-line explanation or a displaying of various loss. That's why verbose will be equal to one.

[27:44] So, let us append all the reconstruction loss, KL loss, one after the other, like a history, so that we have all the five different loss. That's why we make use of this appending function or concatenating. We have one values after the other. So, let us see what is what are the different losses at different training of beta values. So, training beta equal to 0.5. At the end of the fifth epoch, we see that the total loss is 24.0081.

[28:18] In this case, the reconstruction loss would be very minimal. We can see it

here. Yes. We can see the reconstruction loss is 19.02, KL loss is 9.95. Now, let us come to beta equal to one. At the end of fifth epoch, we see that the reconstruction loss is slightly higher when compared to the previous case. So, 19.02 is now increased to 22.35, but KL loss has been reduced 9.95 to 5.55. Now, let us come to next higher value of beta. Beta equal to 2. What has happened is reconstruction loss is 27.79 and KL loss is 1.96.

[29:09] So, it is very much evident that as and when we increase the value of beta, the reconstruction loss increases whereas the KL loss decreases. So, reconstruction will be of poor quality when we increase the value of beta. That's why we should have a we should always try to have a optimal value so that there is a correct balance between the reconstruction loss and the KL loss. So, let us have a comparison of different losses for different beta values for which we are making use of the figure function. Figure size is the argument, 8 in width and 5 in height. So, let us have the plot of different beta values reconstruction losses. Let the marker used be O marker for the reconstruction loss. Same way for different beta

[30:01] values, let us have a plot for KL losses. Let the marker be the square marker. That's why we have this alphabet S here. So, let the X label the label given to the X axis be beta. Let the label given to the Y axis be loss, and let the title for this graph be effect of beta on beta variational autoencoder. Let the legend be present so that we have indication for different types of different values of beta or indication for what what color indicates which loss.

[30:35] So, let the legend be present. Let the grid be also present. That is this the set of gray lines or the square boxes what we have. So, let us show the graph. So, what is seen in the graph? On the title we have effect of beta on beta variational autoencoder. Uh on the X label we have beta, on the Y label we have um loss. So, blue color graph indicates the reconstruction loss. So, beta varying from 0 to 10. As beta increases, the reconstruction loss has increased.

[31:10] So, this is what we have seen uh after every epoch. Same way, as beta is increased, the KL loss is decreasing. So, that there should be a proper balance between these two losses. So, that can be very challenging at times. So, let us uh come to the last code cell wherein we are trying to reconstruct one digit. So, let us take one uh sample. X_{test} of 0 is to 1. So, one sample. Uh zeroth index sample, let us take. Let us have a figure with uh different axis.

[31:49] So, subplots is based on the length of the beta value. So, we we need five different beta values. That's why length of the beta values will be equal to five plus one. Let us have a figure size of 15 uh inches in width and 3 inches in height. So, what we are trying to do is we have this encoder which is doing a prediction on the sample that we are taking. Now, what is the sample that we are taking? X_{test} of 0. Same thing. Prediction on Z. Z is made up of mu and the variance. So, let us

show this reconstructed image. So, let us reshape this reconstructed image which is having the shape of 784 variables into 28 and 28. 28 inches in or 28 pixels in width and 28 pixels in height. Let the color map be equal to

[32:45] gray. We can set any title, say beta, and axis is not required. That's why we are keeping it off. So, let us see how the reconstruction has taken place. So, original image was seven, the label was seven. When beta is equal to 0.5, reconstruction has happened here. Similarly, beta equal to one, a kind of reconstruction, which is which we are still able to predict the number. But, as and when beta is increased, the reconstruction quality has become very poor. We are not able to identify the digits properly, or maybe when coming to beta equal to 10, I think so it is not even seven. The digit is something else, maybe three or nine. So, beta, as and when we increase the reconstruction, has come down. The

[33:38] the quality has become is degraded. Whereas, coming to the latent space, we have a lot of latent space, or more disentanglement when we increase the beta. So, getting a right value of beta is very much important in case of beta variational autoencoder. So, in today's hands-on, we have seen the beta variational autoencoder, which is a type of variational autoencoder. Beta is a hyperparameter, which helps in balancing between the reconstruction loss and the KL divergence loss. So, in the hands-on, we have seen as beta value increases, the reconstruction quality degrades. So, beta value has to be one optimal value, so that we have a proper balance between these two losses.

[34:29] But, Uh, when we see or when we have done enough research on this beta variational autoencoder, we have always seen that the performance of beta VAE is always better than the traditional VAE because in case of beta VAE, the distribution, the latent space distribution is almost nearer to the prior distribution just like a traditional VAE. Along with that, the complex patterns in the data can also be determined. So, that's why it is helpful in a proper reconstruction of the data provided we have optimal beta values.

[35:06] Thank you. »